

条, 涉及最多 50 条 Student 记录, 因此读取 Student 表的块数也可大大减少。

这个简单的例子充分说明了查询优化的必要性, 同时也给出一些查询优化方法的初步概念。例如, 读者可能已经发现, 在第一种情况下连接后的元组可以先不立即写出, 而是和第②步的选择操作结合, 这样可以省去写出和读入的开销。有选择和连接操作时, 可以先做选择操作, 例如把上面的代数表达式 Q_1 、 Q_2 变换为 Q_3 , 这样参加连接的元组就可以大大减少, 这是代数优化。在 Q_3 中, SC 表的选择操作算法可以采用全表扫描或索引扫描, 经过初步估算, 索引扫描方法较优。同样, 对于 Student 和 SC 表的连接, 利用 Student 表上的索引, 采用索引连接代价也较小, 这就是物理优化。

10.3 代数优化

SQL 语句经过查询分析和查询检查后变换为语法树, 它是关系代数表达式的内部表示。本节介绍基于关系代数等价变换规则的优化方法, 即代数优化, 也称作逻辑优化。

10.3.1 关系代数表达式等价变换规则

代数优化策略是通过对关系代数表达式的等价变换来提高查询效率。所谓关系代数表达式的等价, 是指用相同的关系统代替两个表达式中相应的关系所得到的结果是相同的。两个关系表达式 E_1 和 E_2 是等价的, 可记为 $E_1 \equiv E_2$ 。

下面是常用的等价变换规则, 证明从略。

1. 连接运算、笛卡儿积运算的交换律

设 E_1 和 E_2 是关系代数表达式, F 是连接运算的条件, 则有

$$\begin{aligned} E_1 \times E_2 &\equiv E_2 \times E_1 \\ E_1 \bowtie E_2 &\equiv E_2 \bowtie E_1 \\ E_1 \bowtie_F E_2 &\equiv E_2 \bowtie_F E_1 \end{aligned}$$

2. 连接运算、笛卡儿积运算的结合律

设 E_1 、 E_2 、 E_3 是关系代数表达式, F_1 和 F_2 是连接运算的条件, 则有

$$\begin{aligned} (E_1 \times E_2) \times E_3 &\equiv E_1 \times (E_2 \times E_3) \\ (E_1 \bowtie E_2) \bowtie E_3 &\equiv E_1 \bowtie (E_2 \bowtie E_3) \\ (E_1 \bowtie_{F_1} E_2) \bowtie_{F_2} E_3 &\equiv E_1 \bowtie_{F_1} (E_2 \bowtie_{F_2} E_3) \end{aligned}$$

3. 投影运算的串接律

$$\Pi_{A_1, A_2, \dots, A_n} (\Pi_{B_1, B_2, \dots, B_m} (E)) \equiv \Pi_{A_1, A_2, \dots, A_n} (E)$$

其中, E 是关系代数表达式, $A_i (i=1, 2, \dots, n)$, $B_j (j=1, 2, \dots, m)$ 是属性名, 且 $\{A_1, A_2, \dots, A_n\}$ 是 $\{B_1, B_2, \dots, B_m\}$ 的子集。

4. 选择运算的串接律

$$\sigma_{F_1} (\sigma_{F_2} (E)) \equiv \sigma_{F_1 \wedge F_2} (E)$$

其中, E 是关系代数表达式, F_1 、 F_2 是选择条件。选择运算的串接律说明选择条件可以合并, 这样一次就可检查全部条件。

5. 选择运算与投影运算的交换律

$$\sigma_F(\Pi_{A_1, A_2, \dots, A_n}(E)) \equiv \Pi_{A_1, A_2, \dots, A_n}(\sigma_F(E))$$

其中, 选择条件 F 只涉及属性 $A_1, A_2, \dots, A_n$ 。

若 F 中有不属于 $A_1, A_2, \dots, A_n$ 的属性 $B_1, B_2, \dots, B_m$, 则有更一般的规则:

$$\Pi_{A_1, A_2, \dots, A_n}(\sigma_F(E)) \equiv \Pi_{A_1, A_2, \dots, A_n}(\sigma_F(\Pi_{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m}(E)))$$

6. 选择运算与笛卡儿积运算的交换律

如果 F 中涉及的属性都是 E_1 中的属性, 则

$$\sigma_F(E_1 \times E_2) \equiv \sigma_F(E_1) \times E_2$$

如果 $F = F_1 \wedge F_2$, 并且 F_1 只涉及 E_1 中的属性, F_2 只涉及 E_2 中的属性, 则由上面的等价变换规则 1、4、6 可推出:

$$\sigma_F(E_1 \times E_2) \equiv \sigma_{F_1}(E_1) \times \sigma_{F_2}(E_2)$$

若 F_1 只涉及 E_1 中的属性, F_2 涉及 E_1 和 E_2 两者的属性, 则仍有

$$\sigma_F(E_1 \times E_2) \equiv \sigma_{F_2}(\sigma_{F_1}(E_1) \times E_2)$$

它使部分选择在笛卡儿积前先做。

7. 选择运算与并运算的分配律

设 E_1 、 E_2 有相同的属性名, 则

$$\sigma_F(E_1 \cup E_2) \equiv \sigma_F(E_1) \cup \sigma_F(E_2)$$

8. 选择运算与差运算的分配律

若 E_1 与 E_2 有相同的属性名, 则

$$\sigma_F(E_1 - E_2) \equiv \sigma_F(E_1) - \sigma_F(E_2)$$

9. 选择运算对自然连接运算的分配律

$$\sigma_F(E_1 \bowtie E_2) \equiv \sigma_F(E_1) \bowtie \sigma_F(E_2)$$

F 只涉及 E_1 与 E_2 的公共属性。

10. 投影运算与笛卡儿积运算的分配律

设 E_1 和 E_2 是两个关系表达式, $A_1, A_2, \dots, A_n$ 是 E_1 的属性, $B_1, B_2, \dots, B_m$ 是 E_2 的属性, 则

$$\Pi_{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m}(E_1 \times E_2) \equiv \Pi_{A_1, A_2, \dots, A_n}(E_1) \times \Pi_{B_1, B_2, \dots, B_m}(E_2)$$

11. 投影运算与并运算的分配律

设 E_1 和 E_2 有相同的属性名, 则

$$\Pi_{A_1, A_2, \dots, A_n}(E_1 \cup E_2) \equiv \Pi_{A_1, A_2, \dots, A_n}(E_1) \cup \Pi_{A_1, A_2, \dots, A_n}(E_2)$$

10.3.2 语法树的启发式优化

本小节讨论应用启发式规则 (heuristic rule) 的代数优化, 这是对关系代数表达式的语法树

进行优化的方法。典型的启发式规则有：

① 选择运算应尽可能先做。在优化策略中这是最重要、最基本的一条。它常常可使执行代价节约几个数量级，因为选择运算一般使计算的中间结果大大变小。

② 投影运算和选择运算应同时进行。如有若干投影和选择运算，并且它们都对同一个关系操作，则可以在扫描此关系的同时完成所有这些运算，以避免重复扫描关系。

③ 把投影同其前或其后的双目运算结合起来，没有必要为了去掉某些字段而扫描一遍关系。

④ 把某些选择与在它前面要执行的笛卡儿积结合起来成为一个连接运算，连接（特别是等值连接）运算要比同样关系上的笛卡儿积省很多时间。

⑤ 找出公共子表达式。如果这种重复出现的子表达式的结果不是很大的关系，并且从外存中读入这个关系比计算该子表达式的时间少得多，则先计算一次公共子表达式并把结果写入中间文件是合算的。当查询的对象是视图时，定义视图的表达式就是一种公共子表达式。

下面给出遵循这些启发式规则，应用关系代数等价变换规则来优化关系表达式的算法。

算法：关系表达式的优化。

输入：一个关系表达式的语法树。

输出：优化的语法树。

方法：

① 利用等价变换规则 4，把形如 $\sigma_{F_1 \wedge F_2 \wedge \dots \wedge F_n}(E)$ 的表达式变换为 $\sigma_{F_1}(\sigma_{F_2}(\dots(\sigma_{F_n}(E))\dots))$ 。

② 对每一个选择，利用等价变换规则 4~9，尽可能把它移到树的叶端。

③ 对每一个投影，利用等价变换规则 3、5、10、11 中的一般形式，尽可能把它移向树的叶端。

注意：等价变换规则 3 使一些投影消失，而等价规则 5 把一个投影分裂为两个，其中一个有可能被移向树的叶端。

④ 利用等价变换规则 3~5，把选择和投影的串接合并成单个选择、单个投影或一个选择后跟一个投影，使多个选择或投影能同时执行，或在一次扫描中全部完成。尽管这种变换似乎违背“投影尽可能早做”的原则，但这样做效率更高。

⑤ 把经过上述变换得到的语法树的内结点分组。每一双目运算 ($\times$, $\bowtie$, $\cup$, $-$) 和它所有的直接祖先为一组（这些直接祖先是单目运算 σ 或 Π ）。如果其后代直到叶结点全是单目运算，则也将它们并入该组，但当双目运算是笛卡儿积 ($\times$)，而且后面不是与它组成等值连接的选择时，则不能把选择与这个双目运算组成同一组。把这些单目运算单独分为一组。

[例 10.4] 给出例 10.3 中 SQL 语句的代数优化。

```
SELECT Student.Sname FROM Student, SC
WHERE Student.Sno=SC.Sno AND SC.Cno='81003';
```

① 把 SQL 语句转换成语法树,如图 10.3 所示。为了使用关系代数表达式的优化法,不妨假设内部表示是关系代数语法树,如图 10.4 所示。

② 对语法树进行优化。利用等价变换规则 4、6 把选择 $\sigma_{SC.Cno='81003'}$ 移到叶端,则图 10.4 所示的语法树便转换成图 10.5 所示优化后的语法树。这就是例 10.3 中 Q_3 的语法树表示。前面已经分析了 Q_3 比 Q_1 、 Q_2 的查询效率要高得多。

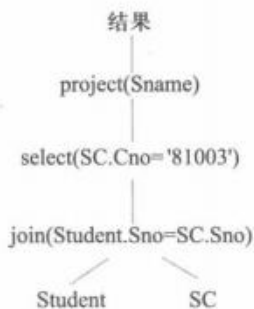


图 10.3 语法树

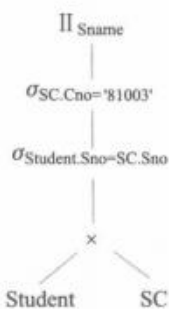


图 10.4 关系代数语法树

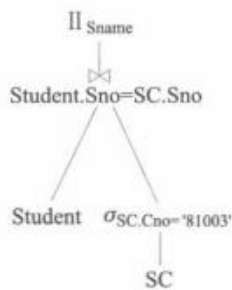


图 10.5 优化后的语法树

10.4 物理优化

代数优化改变查询语句中操作的次序和组合,但不涉及底层的存取路径。10.1.2 小节已讲解了对每一种操作有多种执行这个操作的算法,有多条存取路径,因此对于一个查询语句有多种存取方案,它们的执行效率不同,有的会相差很大。因此,仅仅进行代数优化是不够的。物理优化就是要选择高效合理的操作算法或存取路径,求得优化的查询计划,最终达到查询优化的目标。

选择的方法可以是:

① 基于启发式规则的优化。启发式规则是指那些在大多数情况下都适用,但在不是每种情况下都是最好的规则。

② 基于代价估算的优化。使用优化器估算不同执行策略的代价,并选出具有最小代价的执行计划。

③ 两者结合的优化方法。查询优化器通常会把这两种技术结合在一起使用。因为可能的执行策略很多,要穷尽所有的策略进行代价估算往往是不可行的,会导致查询优化本身付出的代价大于获得的益处。为此,常常先使用启发式规则选取若干较优的候选方案,减少代价估算的工作量,然后分别计算这些候选方案的执行代价,较快地选出最终的优化方案。

10.4.1 基于启发式规则的优化

下面以选择操作和连接操作为例,介绍常用的启发式规则。